

Plan de tests & jeu d'essai

Application web « Petit Rayon de Soleil ». Tests automatisés de l'API REST couvrant les parcours critiques du cahier des charges : authentification, contrôle d'accès, modération et conformité RGPD.

Projet petit-rayon-de-soleil v1.0.0 **Suite** tests/api.test.js **Commande** npm test

Date d'exécution 12/07/2026

7 / 7

Tous les tests passent.

0 échec · 0 ignoré · durée totale 1 745 ms

01 Stratégie de test

La stratégie repose sur des **tests d'intégration automatisés** exécutés de bout en bout contre le serveur réel : chaque test démarre l'application Express, envoie de vraies requêtes HTTP via `fetch`, et vérifie le code de statut et le corps de la réponse. Cette approche valide simultanément le routage, la logique métier, l'accès aux données SQLite et la sécurité — au plus près de l'usage réel.

Le choix a été fait de couvrir en priorité les exigences **Must** du MoSCoW et les exigences non fonctionnelles de sécurité, plutôt que de viser une couverture exhaustive de chaque route :

- **Cas nominal** — un message aléatoire est bien renvoyé (page d'accueil).
- **Authentification** — inscription puis connexion valides ; connexion refusée avec un mauvais mot de passe.
- **Contrôle d'accès** — une route protégée refuse l'anonyme (401) ; une route admin refuse un utilisateur normal (403).
- **Parcours métier complet** — proposition d'un message → invisible tant qu'il n'est pas approuvé → approbation admin → visible.
- **Conformité RGPD** — la suppression de compte efface l'utilisateur, qui ne peut plus se connecter.

Outil retenu : le module natif `node:test` (assertions `node:assert`), sans aucune dépendance de test à installer. Cohérent avec la philosophie minimale du projet et exécutable partout où Node est présent.

02 Environnement de test

Les tests s'exécutent en local, sur une **base de données isolée**. Le hook `before()` supprime l'éventuel fichier `soleil.db` puis le recrée et le remplit (seed) avant la première assertion ; le serveur écoute sur un port dédié `3999` pour ne pas entrer en conflit avec une instance de développement.

Système	Windows 10 (win32 x64)
Runtime	Node.js v24.12.0
Gestionnaire	npm 11.6.2
Base de données	SQLite via node:sqlite (natif)
Port de test	3999 (isolé du port applicatif 3000)
Framework de test	node:test + node:assert
Jeu de données	seed : 1 admin, 1 utilisateur, 5 messages

Procédure d'exécution

- 1 Installer les dépendances applicatives : `npm install` .
- 2 S'assurer qu'aucune instance du serveur ne tient `soleil.db` ouverte (sinon le verrou de fichier empêche la réinitialisation de la base sous Windows).
- 3 Lancer la suite : `npm test` (raccourci de `node --test`).

03 Résultats : attendu vs obtenu

Chaque ligne correspond à un test de `tests/api.test.js`. La colonne « Obtenu » reporte le résultat réel de l'exécution du 12/07/2026.

Tableau de synthèse des 7 tests — entrée, résultat attendu, résultat obtenu.

#	Scénario	Requête	Attendu	Obtenu
1	Message aléatoire (accueil)	GET /api/messages/random	200 + contenu non vide	✓ 200
2	Inscription puis connexion	POST /api/register → POST /api/login	201 puis 200, jeton émis	✓ 201 / 200
3	Connexion, mauvais mot de passe	POST /api/login	401 (refus)	✓ 401
4	Proposition sans authentification	POST /api/messages	401 (non authentifié)	✓ 401
5	Accès admin par un utilisateur normal	GET /api/admin/pending	403 (interdit)	✓ 403
6	Parcours proposition → approbation	POST /api/messages → POST /api/admin/approve/:id	invisible avant, 200, visible après	✓ Conforme
7	Suppression de compte (RGPD)	DELETE /api/me	200, reconnexion impossible (401)	✓ 200 / 401

Capture de la sortie du terminal

petit-rayon-de-soleil – npm test

```
$ npm test
```

```
> petit-rayon-de-soleil@1.0.0 test
```

```
> node --test
```

Base initialisée avec un admin, un utilisateur et 5 messages.

- ✓ GET /api/messages/random renvoie un message (144.66ms)
- ✓ inscription puis connexion fonctionnent (293.67ms)
- ✓ connexion avec mauvais mot de passe est refusée (119.81ms)
- ✓ proposer un message sans être connecté est refusé (401) (4.42ms)
- ✓ un utilisateur normal ne peut pas accéder à l'admin (403) (120.08ms)
- ✓ parcours complet : proposition -> approbation admin (245.25ms)
- ✓ suppression de compte (RGPD) efface l'utilisateur (125.88ms)

```
i tests 7
```

```
i suites 0
```

```
i pass 7
```

```
i fail 0
```

```
i skipped 0
```

```
i duration_ms 1745.69
```

04 Jeu d'essai détaillé — fonctionnalité la plus représentative

La fonctionnalité retenue est le **parcours proposition** → **approbation** (test #6). Elle traverse toutes les couches de l'application et implique les deux rôles (utilisateur et administrateur), ce qui en fait le scénario le plus démonstratif du bon fonctionnement du système.

Décomposition étape par étape : entrée → attendu → obtenu → écart.

Étape	Entrée	Attendu	Obtenu	Écart
1 · Proposer	Utilisateur damien connecté envoie POST /api/messages avec { content: "Tu es capable de grandes choses." }	201, message créé avec approved = 0	✓ 201	—
2 · Vérifier l'invisibilité	GET /api/messages (liste publique)	le message n'apparaît pas	✓ absent	—
3 · Approuver	Admin connecté envoie POST /api/admin/approve/:id	200, passage à approved = 1	✓ 200	—
4 · Vérifier la visibilité	GET /api/messages à nouveau	le message apparaît désormais	✓ présent	—

Aucun écart constaté. Le comportement observé correspond exactement au comportement attendu à chaque étape. Le cycle de modération fonctionne : un message reste privé tant qu'un administrateur ne l'a pas approuvé.

Piste d'amélioration

Pour renforcer le volet sécurité du plan de tests, un test explicite d'anti-injection SQL pourrait être ajouté (envoi d'une charge du type ' OR '1'='1 dans un champ, et vérification qu'elle est traitée comme une donnée littérale grâce aux requêtes paramétrées, sans effet sur la base).